

KANTIPUR ENGINEERING COLLEGE

(Affiliated to Tribhuvan University)

Dhapakhel, Lalitpur



[Subject Code: CT 755]

A MAJOR PROJECT FINAL REPORT ON NEPALI HATE TEXT DETECTION AND FILTERING SYSTEM

Submitted by:

NISHAN PATHAK [KAN078BCT050]

PRAJAKTA CHALISE [KAN078BCT055]

RIYA BHATTARAI [KAN078BCT065]

SUBIJ DHITAL [KAN078BCT087]

**A MAJOR PROJECT SUBMITTED IN PARTIAL
FULFILLMENT OF THE REQUIREMENT FOR THE DEGREE
OF BACHELOR IN COMPUTER ENGINEERING**

Submitted to:

Department of Computer and Electronics Engineering

March, 2026

NEPALI HATE TEXT DETECTION AND FILTERING SYSTEM

Submitted by:

NISHAN PATHAK [KAN078BCT050]

PRAJAKTA CHALISE [KAN078BCT055]

RIYA BHATTARAI [KAN078BCT065]

SUBIJ DHITAL [KAN078BCT087]

Supervised by:

Prof. Dr. Subarna Shakya

Institute of Engineering, Tribhuvan University

Pulchowk, Lalitpur, Nepal

**A MAJOR PROJECT SUBMITTED IN PARTIAL
FULFILLMENT OF THE REQUIREMENT FOR THE DEGREE
OF BACHELOR IN COMPUTER ENGINEERING**

Submitted to:

Department of Computer and Electronics Engineering

Kantipur Engineering College

Dhapakhel, Lalitpur

March, 2026

COPYRIGHT

The author has agreed that the library, Kantipur Engineering Collage, may make this report freely available for inspection. Moreover the author has agreed that permission for extensive copying of this report for scholarly purpose may be granted by the supervisor(s), who supervised the project work recorded herein or, in their absence, by the Head of the Department wherein this project was done. It is understood that due recognition will be given to the author of this report and to the Department of Computer and Electronics Engineering, Kantipur Engineering College in any use of the material of this report. Copying or publication or other use of this report for financial gain without approval of the Department of Computer and Electronics Engineering, Kantipur Engineering College and author's written permission is prohibited.

Request for permission to copy or to make any other use of the material in this report in whole or in part should be addressed to:

Head

Department of Computer and Electronics Engineering

Kantipur Engineering College

Dhapakhel, Lalitpur

Nepal

KANTIPUR ENGINEERING COLLEGE
DEPARTMENT OF COMPUTER AND ELECTRONICS ENGINEERING
APPROVAL LETTER

The undersigned certify that they have read and recommended to the Institute of Engineering for acceptance, a project report entitled "Nepali Hate Text Detection and Filtering System" submitted by

NISHAN PATHAK [KAN078BCT050]

PRAJAKTA CHALISE [KAN078BCT055]

RIYA BHATTARAI [KAN078BCT065]

SUBIJ DHITAL [KAN078BCT087]

in partial fulfillment for the degree of Bachelor in Computer Engineering.

.....

Supervisor

Prof. Dr. Subarna Shakya

Institute of Engineering, Tribhuvan University

Pulchowk, Lalitpur, Nepal

Date: March 29, 2026

ABSTRACT

Hate text is increasing on Nepali online platforms such as social media and various news portal websites. Due to the lack of strong tools made specifically for the Nepali language, hate and offensive comments often go unnoticed. Most available tools are designed for English and do not understand Nepali, mixed languages or cultural nuances. This project presents a hate text detection and filtering system specifically designed for the Nepali digital space using a fine-tuned NepBERTa model trained on Devanagari script. A dataset of over 39000 samples was collected from four distinct sources including social media posts, Kaggle, GitHub and YouTube-scraped content and labeled into hate and non-hate categories. Class imbalance between non-hate and hate samples was addressed using class weights. The system followed a structured pipeline consisting of text pre-processing, tokenization, classification and text masking. Five variants of NepBERTa were evaluated, with NepBERTa-V5 achieving the best performance with macro F1-score of 84.33%. The trained model was deployed as a Flask-based web application that identifies potentially offensive Nepali text and filters such hate content in the user interface. This implementation demonstrates a practical tool for assisting automated content filtering which serves as lightweight moderation aid in the Nepali language ecosystem.

Keywords: Hate text detection, Nepali NLP, NepBERTa, BERT, Devanagari script, class imbalance

ACKNOWLEDGMENT

We would like to express our gratitude to everyone who helped us complete this project. First and foremost, we would like to acknowledge the crucial role of our teachers in the Department of Electronics and Computer Engineering for their guidance, support, and feedback throughout the project. We extend our sincere appreciation to Er. Rabindra Khati, Head of the Department of Computer and Electronics, for his valuable support. Additionally, we are grateful to Er. Aliz Shrestha and Er. Sahaj Shakya for their continuous help and encouragement during the course of our project.

Next, we would like to express our gratitude to our classmates for providing constructive feedback and engaging in thought-provoking discussions regarding our project. We also extend our thanks to all the lecturers of our department for guiding us from the beginning to the completion of this project.

Finally, a special gratitude to our family and friends for their love, encouragement, and support throughout our academic journey. Thank you all for your invaluable contributions to this project.

NISHAN PATHAK	[KAN078BCT050]
PRAJAKTA CHALISE	[KAN078BCT055]
RIYA BHATTARAI	[KAN078BCT065]
SUBIJ DHITAL	[KAN078BCT087]

CONTENTS

Copyright	i
Approval Letter	ii
Abstract	iii
Acknowledgment	iv
Abbreviations	v
1 Introduction	1
1.1 Background	1
1.2 Problem Statement	2
1.3 Objectives	2
1.4 Applications and Social Impact	2
1.5 Project Features	3
1.6 Feasibility Analysis	3
1.6.1 Economic Feasibility	3
1.6.2 Technical Feasibility	3
1.6.3 Operational Feasibility	4
1.6.4 Schedule Feasibility	4
1.7 System Requirements	5
1.7.1 Development Requirements	5
1.7.2 Deployment Requirements	6
2 Literature Review	7
2.1 Related Research	7
3 Theoretical Foundation	11
3.1 NLP	11
3.1.1 Definition of NLP	11
3.1.2 NLP Challenges in Nepali	11
3.2 Hate Text Detection as a Classification Task	11

3.2.1	Text Classification	11
3.2.2	Working of Text Classification	12
3.2.3	Nepali Hate Text Detection	12
3.3	Transformer-Based Language Models	13
3.3.1	Transformer Architecture	13
3.3.2	Self-Attention Mechanism	14
3.3.3	Multi-Head Attention	14
3.3.4	Positional Encoding	14
3.3.5	Position-wise Feed-Forward Networks	14
3.3.6	Embeddings	15
3.3.7	Encoder–Decoder Architecture	15
3.3.8	Softmax Layer for Output Prediction	16
3.4	BERT-Based Models	17
3.4.1	BERT Overview	17
3.4.2	BERT Architecture	17
3.4.3	Pre-training of BERT	18
3.4.4	Fine-Tuning	18
3.4.5	Fine-Tuning for Text Classification	18
3.5	NepBERTa Model	19
3.5.1	Overview of NepBERTa	19
3.5.2	NepBERTa Base Configuration and Operation	19
3.6	Cross-Entropy Loss	23
3.6.1	Binary Cross-Entropy Loss	23
3.7	Model Evaluation Metrics	24
4	Methodology	26
4.1	System Overview	26
4.2	System Diagrams	29
4.2.1	Use Case Diagram	29
4.2.2	Sequence Diagram	30
4.2.3	Activity Diagram	31
4.2.4	Software Development Lifecycle Model	32

5	RESULT AND DISCUSSIONS	33
5.0.1	Data Collection	33
5.0.2	Data preprocessing and cleaning	34
5.0.3	Data Labeling	35
5.0.4	Model Evaluation and Selection	35
5.0.5	Training of the Selected Model NepBERTa- V5	36
5.0.6	Model Inference and Error Analysis	40
6	Conclusion and future enhancements	42
6.1	Conclusion	42
6.2	Future enhancements	42
	REFERENCES	43
A	Standardized Data Annotation Guidelines	46
A.1	Introduction	46
A.2	Annotation Categories	46
A.2.1	Category 0: NEUTRAL / NON-HATE	46
A.2.2	Category 1: HATE	47
A.3	Annotation Guideline	48
A.3.1	Annotation Visualization	48
A.3.2	Personal attack vs Criticism	48
A.4	Annotation examples	49
A.5	Kappa Score	49
A.5.1	Inter-Annotator Agreement using Fleiss' Kappa	50
B	Output	52

LIST OF FIGURES

1.1	Gantt Chart	4
3.1	Transformer encoder-decoder architecture	13
3.2	Architecture diagram of BERT with different sizes BERT _{BASE} and BERT _{LARGE}	17
3.3	Pre-training and fine-tuning of NepBERTa	23
4.1	System Block Diagram	26
4.2	Use Case Diagram	29
4.3	Sequence Diagram	30
4.4	Activity Diagram	31
4.5	Incremental Model	32
5.1	Pre-processing	34
5.2	Training vs Validation Graphs	38
5.3	Training vs Validation Graphs	39
5.4	Confusion Matrix	40
A.1	Guideline Visualization	48
B.1	Output 1	52
B.2	Output 2	52
B.3	Output 3	53

LIST OF TABLES

1.1	Development Requirements	5
1.2	Deployment Requirements	6
5.1	Baselining of NepBERTa	36
A.1	Labeling Examples	49

LIST OF ABBREVIATIONS

NLP	Natural Language Processing
BERT	Bidirectional Encoder Representations from Transformers
API	Application Programming Interface
UML	Unified Modeling Language
MLM	Masked Language Modeling
URL	Uniform Resource Locator
NepBERTa	Nepali Bidirectional Encoder Representations from Transformers architecture

CHAPTER 1

INTRODUCTION

1.1 Background

The rapid growth of social media has significantly changed how people communicate around the world. As of 2026, there are more than 5.8 billion users globally including approximately 18.3 million users in Nepal. While these platforms have created new opportunities for connection and free expression, they have also made it easier for harmful content such as hate speech, harassment and offensive language to spread rapidly. Such content can negatively impact individuals and disturb social harmony. It is important to detect and control it in a timely manner.

Content moderation in the Nepali context is particularly challenging due to the lack of language-specific tools. Most existing moderation systems are designed primarily for English and often fail to properly understand Nepali text written in the Devanagari script. As a result, harmful messages in Nepali frequently go undetected. Traditional machine learning approaches such as Naive Bayes rely heavily on word frequency and are limited in their ability to capture the deeper meaning and context of sentences.

To address this limitation, transformer-based models like BERT are more effective as they analyze text contextually by considering both preceding and following words. Since the original BERT model is trained on English data this project uses NepBERTa, a pre-trained language model specifically designed for Nepali. NepBERTa is further fine-tuned using a custom dataset of Nepali social media text to improve its ability to detect hate speech within the local linguistic and cultural context.

This project aims to develop a hate text detection system tailored for the Nepali digital space. In addition to identifying harmful content, the system provides basic filtering functionality by hiding or masking detected hate text within the user interface. This approach helps reduce the visibility of offensive content and supports the creation of a safer and more positive online environment, particularly for younger users and marginalized communities. However, the system is intended as a lightweight moderation aid and does

not implement a complete content moderation pipeline.

1.2 Problem Statement

Hate text is defined as a language that is used to express hatred towards a targeted group or is intended to be derogatory, humiliating or insult to the members of marginalized groups. Hate text is increasing in Nepali online platforms like social media and various news portal websites. Due to lack of strong tools made specifically for Nepali language, hate and offensive comments often go unnoticed. Most of the tools which are available are made for English and don't understand Nepali, mixed languages or cultural nuances. The main reasons for this problem are lack of labeled Nepali data, weak language models and no system that can detect hate text in Nepali.

1.3 Objectives

1. To develop a system to detect Nepali Devanagari hate text and implement filtering techniques.
2. To create an easily installable and usable python package, allowing users to leverage Nepali Devanagari hate text detection seamlessly.

1.4 Applications and Social Impact

The proposed hate text detection system can be used in social media platforms to support user safety by automatically identifying hate speech in Nepali text. By detecting and filtering potentially harmful content, the system helps reduce the visibility of offensive language. This is particularly useful in environments with large volumes of user-generated content where manual moderation alone is not sufficient.

The system contributes to improving the online experience of Nepali-speaking communities by enabling basic content filtering. By filtering the detected hate text, it can assist in creating a more respectful digital space and encourage healthier interactions among users. However, the system is designed as a lightweight moderation aid and does not implement a full-scale moderation pipeline such as policy enforcement, user reporting,

or administrative control mechanisms.

1.5 Project Features

Due to the absence of efficient tools for detecting hate in Nepali Devanagari text, this project focuses on building a dedicated detection system for Nepali content. The solution is implemented as an installable Python package with an accompanying web-based demonstration interface. The system processes Nepali text inputs, performs hate detection and outputs the results by filtering the detected hate content without applying any form of text modification or neutralization.

1.6 Feasibility Analysis

Starting a new project required a suitable environment along with extensive verification and validation. To ensure this, a feasibility study played a vital role in analyzing whether developing a new system would be beneficial, time-efficient, cost-effective, and aligned with current technologies. The feasibility study was conducted to evaluate these factors before proceeding with the system development.

1.6.1 Economic Feasibility

Economic feasibility was the cost and logistical outlook for any business project or endeavor. It was used to determine whether the cost of the prospective project would ultimately be profitable to the company or not. This project required a PC and a good internet connection to perform well. Overall estimation suggested that the benefits of the proposed system would overcome the initial cost and running of the system.

1.6.2 Technical Feasibility

The technical requirements for the proposed system were economical. The technical feasibility study assessed the hardware and software resources needed for system development. Hardware requirements such as a laptop with sufficient RAM and software tools like VS Code, Jupyter Notebook, and PyCharm were considered due to their ac-

cessibility and suitability for development tasks. Additionally, raw materials for the project such as unprocessed datasets were readily available, which supported efficient implementation of the system.

1.6.3 Operational Feasibility

The proposed system was operationally workable, as it featured a user-friendly interface designed for individuals with minimal technical expertise. Users could easily interact with the system to detect hateful or offensive content without requiring extensive training. The system functioned efficiently and aligned with existing content moderation workflows, integrating seamlessly with current platforms or communication tools to minimize disruption during deployment.

1.6.4 Schedule Feasibility

The workload of the project was divided among the project members. Different modules were assigned to group members based on the project schedule. This ensured that the project met the schedule feasibility requirements.

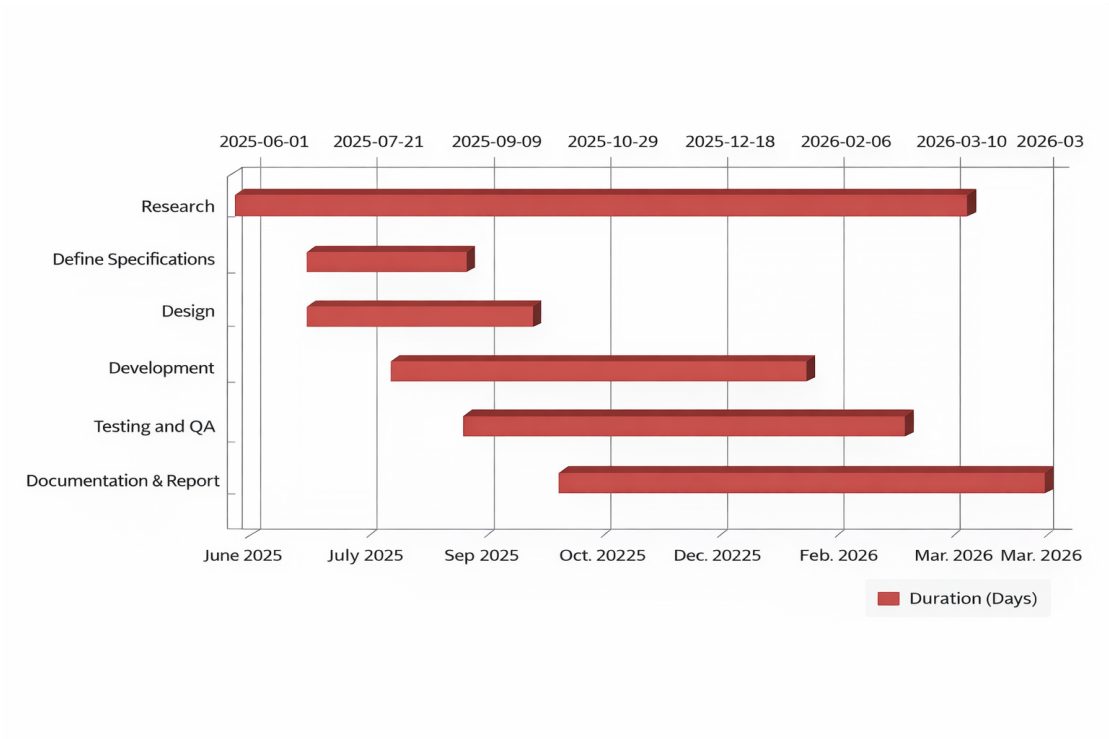


Figure 1.1: Gantt Chart

1.7 System Requirements

1.7.1 Development Requirements

Table 1.1: Development Requirements

Hardware Requirements	Software Requirements
•RAM: 8GB (minimum) •Processor: CPU with four or more cores •Storage: 512GB HDD or SSD (recommended) •GPU: recommended •OS: Windows 7 and above	•Visual Studio Code •PyTorch, nltk •Transformers, Optimum •Python, pip, wheel, twine, setup-tools •pyPI account

1.7.2 Deployment Requirements

Table 1.2: Deployment Requirements

Hardware Requirements	Software Requirements
• RAM: 8GB • Processor: CPU with four or more cores • Storage: 512GB HDD or SSD • GPU: recommended • OS: Windows 7 and above	• Visual Studio Code • PyTorch, nltk • Transformers, Optimum • Python, pip, wheel, twine, setuptools • Frameworks and libraries: - PyTorch, nltk - Transformers, Optimum - Python, pip, wheel, twine, setuptools • pyPI account

CHAPTER 2

LITERATURE REVIEW

2.1 Related Research

Timilsina et al. proposed that the performance of natural language understanding (NLU) in low-resource languages like Nepali has traditionally lagged due to the unavailability of large-scale monolingual datasets and pretrained language models. Although multilingual models such as mBERT and XLM-R have been used, the authors argued that these models struggle to capture the unique linguistic characteristics of Nepali due to their broad multilingual training. To address this, they introduced NepBERTa, a BERT based monolingual language model trained on a corpus of over 0.8 billion words collected from 36 major Nepali news websites, making it the most extensive Nepali corpus to date. Inspired by RoBERTa's training methodology, NepBERTa adopts dynamic masked language modeling without next sentence prediction to enhance semantic learning. The model was evaluated on a newly introduced benchmark, Nep-gLUE, which includes four downstream tasks: Named Entity Recognition (NER), Part-of-Speech Tagging (POS), Content Classification (CC) and Categorical Pair Similarity (CPS). Their model outperformed both multilingual and earlier Nepali-specific models, attaining a macro-F1 score of 93.55 on the Nep-gLUE benchmark. It achieved the highest accuracy in Named Entity Recognition with 91.09 percent and in Part-of-Speech Tagging with 95.56 percent, demonstrating the effectiveness of language specific large-scale pretraining [1].

Pudasaini et al. introduced NepaliBERT, a contextual language model pre-trained on a large-scale Nepali corpus using the BERT architecture. Their work addressed the longstanding lack of high-quality NLP models for the Nepali language, which uses the Devanagari script and presents unique syntactic and semantic complexities. To build the model, they combined over 300,000 news articles from 18 sources with the OSCAR dataset, resulting in a corpus of over 94 million tokens. The authors trained three models like SSabda2Vec, Bakya2Vec and NepaliBERT using Word2Vec, Doc2Vec and BERT architectures respectively and evaluated them using intrinsic (perplexity score) and extrinsic (classification tasks) methods. NepaliBERT achieved a training perplexity score

of 2.31 and a perplexity of 50.10 on unseen text, indicating strong generalization within the language domain. For extrinsic evaluation, they conducted sentiment classification on Nepali COVID-19-related tweets across eight classes using various machine learning classifiers. The highest performance was achieved using Support Vector Machine (SVM) with results showing precision 0.74, recall 0.74, F1-score 0.73 and accuracy 0.73. Similarly, competitive results were observed across other classifiers such as Extra Tree Classifier and Logistic Regression which shows its effectiveness in the real world. This work not only advances NLP research in low-resource languages but also provides publicly available tools for future downstream tasks [2].

Niraula et al. presents a comprehensive corpus-based study on offensive language, linguistic taboos and euphemisms in the Nepali language, offering the first large-scale resource for offense-related NLP in this low-resource setting. The authors collected over 7000 social media posts and manually compiled a dataset of 1077 offensive terms categorized into 18 distinct types including race, religion, sex, politics and body parts. In addition, the article introduces 12 types of euphemism techniques such as synonym replacement, metaphor, circumlocution, child-friendly terms and honorific suffixes that are commonly used to reduce linguistic harshness. The constructed dataset includes both taboo and contextually offensive terms, providing a valuable resource for tasks like offensive content detection and comment moderation in NLP pipelines. While the study does not propose or evaluate a classification model, it provides a publicly available dataset that lays the foundation for future offensive language detection tasks in Nepali. Furthermore, the article highlights the cultural uniqueness of offensive language in Nepali and encourages the development of localized NLP tools sensitive to sociolinguistic norms [3].

Thapa et al. paper introduces NEHATE, a large-scale annotated dataset developed for hate speech detection in the Nepali language, particularly in the context of the 2022 local elections. The dataset comprises 13,505 tweets annotated as either hate or non-hate with further sub-categorization of hate speech into targets: individual, organization and community. The annotation process involved a rigorous three-phase protocol with high inter-annotator agreement, demonstrating the reliability of the labeling. Experiments were conducted using both classical machine learning models such as Naive Bayes,

XGBoost and transformer-based models like NepaliBERT and NepBERTa. Among the tested models, NepBERTa demonstrated the highest performance, with an F1-score of 0.68, while human annotators consistently outperformed automated methods with F1-scores above 0.88. The study also revealed that hate speech tweets tend to be longer and more complex, especially in expressing political dissatisfaction. NEHATE thus provides a strong benchmark for future work in hate speech detection for underrepresented languages [4].

Devlin et al. paper introduces BERT (Bidirectional Encoder Representations from Transformers), a novel deep learning architecture that pre-trains deep bidirectional representations from unlabeled text. Unlike earlier models which were unidirectional or shallowly concatenated, BERT employs a masked language model and next sentence prediction during pre-training, allowing it to deeply understand the context from both directions. Fine-tuning BERT requires minimal architectural changes for a wide range of NLP tasks. Their approach achieved state-of-the-art performance on eleven NLP benchmarks, including 93.2 F1 on SQuAD v1.1 and 86.7% accuracy on MultiNLI. These results highlight the effectiveness of bidirectional pretraining and established BERT as a foundational model in natural language understanding [5].

Singh et al. describes a hate speech detection system that uses advanced deep learning techniques, particularly focusing on textual analysis of multimodal content. The system involves two main tasks: hate speech classification and target identification. For these, a preprocessed set of text-embedded images from the CrisisHateMM dataset is used, where Optical Character Recognition (OCR) extracts text from images, followed by normalization techniques like case-folding and punctuation removal. Several models were tested such as BiLSTM, ALBERT, XLNet-base-cased and XLM-Roberta-base. Among them, XLM-Roberta-base showed superior performance across all evaluation metrics. The evaluation was based on two subtasks: sub-task 1 classified images as hate or non-hate and sub-task 2 as target classification. In sub-task 1, XLM-Roberta-base achieved an F1-score of 84.62% and accuracy of 84.65%, while in sub-task 2, it attained an F1-score of 69.73% and accuracy of 72.31%. These results led to a third place ranking in both tasks during the CASE 2023 shared task. The research suggested further enhancements such as expanding to multimodal approaches, addressing dataset

limitations and extending to multi-language detection for improved generalizability and fairness [6].

Melton et al. present DeL-haTE, a deep learning-based tunable ensemble framework for hate speech detection, specifically designed to address issues such as class imbalance, vocabulary variation and the scarcity of labeled data. The framework integrates CNN and GRU layers to form a robust ensemble classifier which is further enhanced through transfer learning and weak supervision. The study evaluates model performance on both labeled Twitter datasets (HON and OLID) and unlabeled Gab posts, showing that the ensemble approach achieves significant improvements. Specifically, on the HON dataset, DeL-haTE attains a hate recall of 83% and an F1-macro of 65%, outperforming existing state-of-the-art models by 33% in hate recall. Transfer learning techniques were applied by tuning pre-trained classifiers on a small, labeled subset of Gab data, resulting in a 12% average gain in hate recall. Additionally, their weak supervision method, which employs lexicon-based heuristics on unlabeled Gab posts, achieves a hate recall of 67%, further demonstrating the model's ability to generalize in low-resource and evolving online environments. These findings emphasize the potential of ensemble learning combined with transfer and weak supervision for scalable hate speech detection across diverse and dynamic social media platforms [7].

CHAPTER 3

THEORETICAL FOUNDATION

3.1 NLP

3.1.1 Definition of NLP

NLP is a branch of AI that enables computers to understand, interpret and generate human language, bridging the gap between human communication and computer understanding. By leveraging machine learning, deep learning and linguistics, NLP powers applications like chatbots, translation services, sentiment analysis and automated text summarization tasks.

3.1.2 NLP Challenges in Nepali

The key challenges for NLP in Nepali language often encountered are:

- Limited labeled datasets
- Informal and noisy social media texts
- Romanized and devnagari mixed texts
- Morphological richness of Nepali
- Unicode and spelling variations

These challenges motivate the use of pretrained transformer-based models.

3.2 Hate Text Detection as a Classification Task

3.2.1 Text Classification

Text classification assigns predefined labels to text data. It is an AI/NLP process that automatically sorts text into predefined categories (labels) using ML/DL techniques, crucial for organizing or categorizing vast amounts of unstructured data in emails into

spam or not spam, reviews into sentiment analysis, texts into hate or non-hate, etc.

3.2.2 Working of Text Classification

Text is classified on the basis of this process flow:

- **Input:** Includes raw text like sentences, documents, reviews, etc.
- **Preprocessing:** Cleaning and preparing the text data for analysis.
- **Feature Extraction:** Converting text into numerical features eg. using TF-IDF or word embeddings.
- **Model Training:** Using ML algorithms like Naive Bayes or DL models like BERT to learn patterns.
- **Prediction:** Assigning one or more labels to new text based on learned patterns.

3.2.3 Nepali Hate Text Detection

The goal is to automatically detect hateful expressions written in Nepali. Text classification is leveraged for Nepali Hate Detection task. Nepali Hate Text Detection is an active area of NLP primarily focusing on social media data, particularly YouTube during sensitive periods like local elections and political instability in the country. Due to the low-resource nature of the Nepali language, recent approaches have transitioned from traditional Machine Learning models to Transformer-based models providing high performance in identifying hate text in Devanagari script.

Contextual understanding is essential because hate often depends on word's meaning in context and not individual words. Transformer models, particularly multilingual variants like XLM-RoBERTa were widely utilized for Nepali hate tasks by leveraging contextual understanding of the Devanagari script. But, recent works in monolingual transformer models such as NepBERTa have advanced the Nepali hate tasks.

3.3 Transformer-Based Language Models

The transformer model is a type of neural network architecture that excels at processing sequential data and have achieved great performance in the fields of artificial intelligence (AI) such as computer vision, speech recognition and time series forecasting. Despite their versatility, transformer models are also commonly discussed in natural language processing (NLP) use cases such as chatbots, text generation, summarization, question answering and sentiment analysis.

BERT (Bidirectional Encoder Representations from Transformers), an encoder-only model introduced by Google in 2019, was a major landmark in the establishment of transformers and remains the basis of most modern word embedding applications from modern vector databases to Google search.

3.3.1 Transformer Architecture

In 2017 Vaswani et al. published a paper "Attention is All You Need" in which the transformers architecture was introduced. The article explores the architecture, workings and applications of transformers.

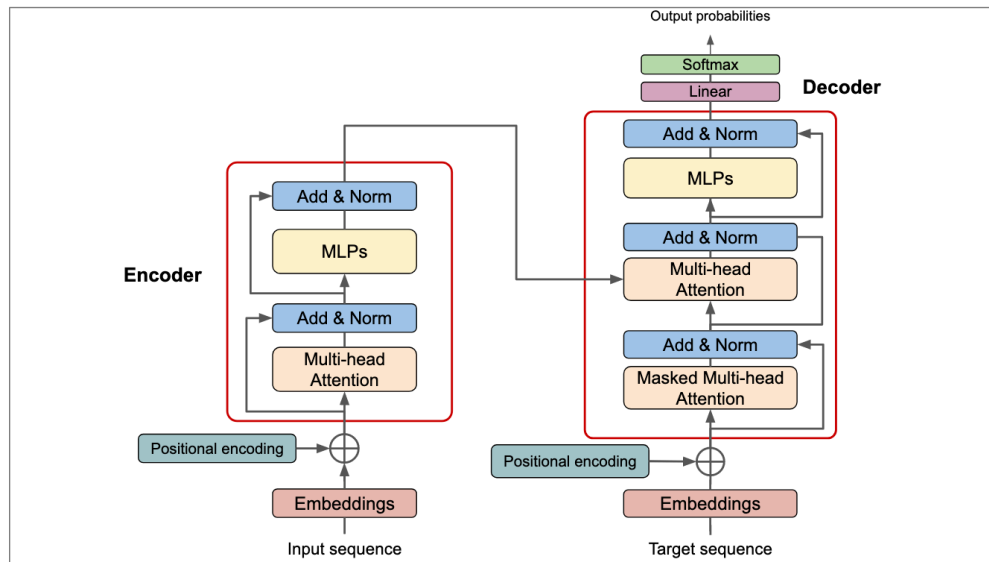


Figure 3.1: Transformer encoder-decoder architecture
Source: <https://deephvision.github.io/posts/001-transformer/>

3.3.2 Self-Attention Mechanism

The self-attention mechanism allows transformers to determine which words in a sentence are most relevant to each other. This is achieved using the scaled dot-product attention approach. Each word in a sequence is mapped to three vectors: Query (Q), Key (K) and Value (V). The attention scores are computed as:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V \quad (3.1)$$

These scores determine how much attention each word should pay to other words in the sequence.

3.3.3 Multi-Head Attention

Instead of using a single attention mechanism, transformers employ multiple attention heads operating in parallel. Each attention head learns different relationships or patterns within the data, thereby enriching the model's representation capability.

3.3.4 Positional Encoding

Unlike recurrent neural networks (RNNs), transformers do not have an inherent understanding of word order because they process tokens in parallel. To incorporate positional information, positional encodings are added to the input embeddings, providing information about the position of each token within the sequence.

3.3.5 Position-wise Feed-Forward Networks

The position-wise feed-forward network consists of two linear transformations with a ReLU activation function in between. This network is applied independently and identically to each position in the sequence.

Mathematically, it is defined as:

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2 \quad (3.2)$$

This transformation helps refine the encoded representation at each position.

3.3.6 Embeddings

Transformers cannot process raw text directly, so each input token, which may be a word or a subword, is first converted into a numerical vector called an embedding. These embeddings represent the meaning of tokens in a form that the model can understand and process. Both encoder and decoder input tokens are transformed into embeddings before being passed through the transformer layers.

The embeddings used in the transformer model are trainable, meaning their values are learned and updated during the training process. This allows the model to capture semantic relationships between tokens based on the training data. In the transformer architecture, the same embedding weight matrix is shared between the encoder embeddings, the decoder embeddings, and the final linear layer that is applied before the softmax function. Weight sharing helps reduce the total number of parameters and improves learning efficiency.

To ensure numerical stability, the embedding vectors are scaled by the square root of the model dimension, denoted as $\sqrt{d_{model}}$. This scaling is applied before adding positional encodings and helps maintain stable value ranges during training. After scaling, positional encodings are added to the embeddings to provide information about the order of tokens in the input sequence, which is necessary because the transformer model itself does not have any built-in notion of sequence order.

3.3.7 Encoder–Decoder Architecture

The encoder–decoder structure is fundamental to transformer models. The encoder processes the input sequence into a contextual representation, while the decoder generates the output sequence based on this representation. For example, a French sentence “*Je suis étudiant*” is translated into “*I am a student*” in English.

Transformers apply attention in three different ways:

Encoder Self-Attention

- Queries, keys, and values all come from the encoder's previous layer.
- Each word can attend to every other word in the input sequence.
- This enables the encoder to capture global contextual information.

Decoder Self-Attention (Masked)

- Queries, keys, and values come from the decoder's previous layer.
- Future tokens are masked so each position can only attend to earlier positions.
- This ensures auto-regressive decoding, where the model predicts one token at a time.

Encoder–Decoder Attention

- Queries come from the decoder.
- Keys and values come from the encoder output.
- This allows the decoder to focus on relevant parts of the input while generating output tokens.

Together, these attention mechanisms allow the transformer to process the entire input sequence while generating outputs step by step with full contextual awareness.

3.3.8 Softmax Layer for Output Prediction

After processing through the decoder, the model predicts the next token in the sequence. The decoder output is passed through a linear layer whose weights are shared with the embedding layer. The softmax function then converts these outputs into probability distributions over the vocabulary. The token with the highest probability is selected as the predicted next word.

3.4 BERT-Based Models

3.4.1 BERT Overview

BERT (Bidirectional Encoder Representations from Transformers) is a transformer-based language model designed to understand contextual relationships in text. Unlike traditional unidirectional language models, BERT processes text bidirectionally, considering both left and right context simultaneously. It uses an encoder-only transformer architecture, focusing on language understanding rather than sequence generation.

3.4.2 BERT Architecture

BERT consists of stacked bidirectional transformer encoders. $\text{BERT}_{\text{BASE}}$ has 12 encoder layers with 768 hidden units and 12 attention heads (110M parameters), while $\text{BERT}_{\text{LARGE}}$ has 24 layers with 1024 hidden units and 16 attention heads (340M parameters). The [CLS] token representation is commonly used for classification tasks.

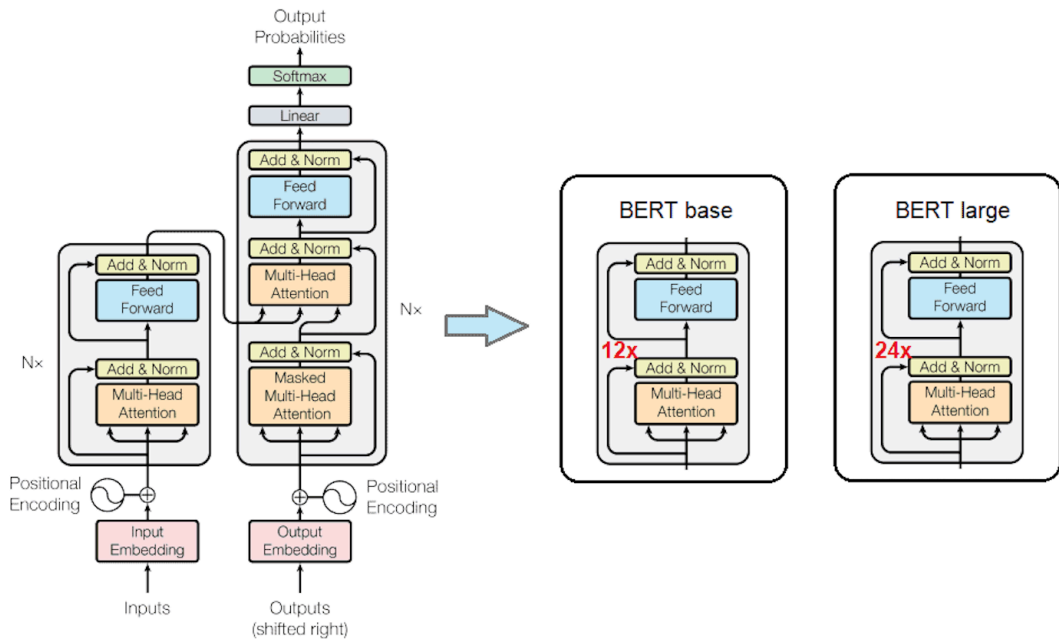


Figure 3.2: Architecture diagram of BERT with different sizes $\text{BERT}_{\text{BASE}}$ and $\text{BERT}_{\text{LARGE}}$

Source: <https://sushant-kumar.com/blog/bert>

3.4.3 Pre-training of BERT

BERT is pre-trained on large-scale unlabeled text to learn contextual word embeddings. It uses two self-supervised training objectives:

- **Masked Language Model (MLM):** Randomly masks a subset of input tokens and predicts them using surrounding context. Loss is computed only on masked tokens.
- **Next Sentence Prediction (NSP):** Predicts whether one sentence logically follows another using the representation of the [CLS] token.

Both objectives are trained jointly to capture intra-sentence context and inter-sentence relationships.

3.4.4 Fine-Tuning

After pre-training, BERT is fine-tuned on labeled data for downstream NLP tasks. A task-specific output layer is added, and all parameters are updated using supervised learning. This unified architecture allows BERT to adapt efficiently to various tasks.

3.4.5 Fine-Tuning for Text Classification

In this project, fine-tuning is performed to adapt the pre-trained BERT-based model to the task of Nepali hate text classification. After pre-training, the model already captures rich contextual representations of the Nepali language, which are leveraged and refined using labeled hate and non-hate text data.

For text classification, a task-specific classification head is added on top of the transformer encoder. The final hidden representation corresponding to the [CLS] token is used as a pooled representation of the entire input text. This vector is passed through a fully connected layer followed by a SoftMax function to predict binary class labels.

During fine-tuning, all model parameters including both the pre-trained encoder and

the classification layer, are updated using supervised learning. The model is trained by minimizing the cross-entropy loss between the predicted labels and the ground-truth annotations. This process enables the model to learn task-specific patterns while retaining its general language understanding.

Fine-tuning allows the model to effectively capture subtle contextual cues, implicit meanings and linguistic variations present in Nepali hate text, leading to improved classification performance compared to training a model from scratch.

3.5 NepBERTa Model

3.5.1 Overview of NepBERTa

NepBERTa is a BERT based language model pre-trained on a large Nepali corpus in Devanagari script. It uses the same underlying architecture as BERT-Base but is optimized entirely for Nepali by being trained on over 0.8 billion words from 36 popular Nepali news websites.

3.5.2 NepBERTa Base Configuration and Operation

1. Number of Layers:

NepBERTa contains 12 transformer encoder layers. Each layer applies attention and feed-forward processing to build rich contextual understanding. These layers enable deep processing of token relationships in long Nepali sentences.

2. Hidden Size:

Each token is represented using a 768 dimensional vector. These embeddings allow the model to retain both meaning and position information about each token in the sentence.

3. Number of Attention Heads:

Each of the 12 layers has 12 parallel attention heads. This means the model can focus on 12 different parts of the sentence simultaneously to learn various patterns like the tone of the message, the target of the sentence and the grammar

relationships.

4. Feed Forward Dimension:

After the attention block, a feed-forward neural network of size 3072 is applied. This increases the expressive power of the model and helps transform the token representations before passing to the next layer.

5. Total Parameters:

NepBERTa has approximately 110 million parameters. These are trainable weights that the model adjusts during training to learn patterns in the Nepali language.

In our proposed system for hate speech detection and correction in Nepali language, the NepBERTa model serves as the core NLP engine. NepBERTa is a monolingual transformer based language model that has been pre-trained on large scale Nepali text corpora written in Devanagari script. It is based on the BERT architecture and has been adapted specifically for low resource languages like Nepali .

NepBERTa works through the following key steps in our hate speech detection and correction project:

Step-1: Input Representation

- **Text Acquisition:** User generated text in pure Nepali (Devanagari script) is input to the system. This text may come from social media posts, comment sections or online forums.
- **Tokenization:** The raw Nepali text is first processed using the SentencePiece Tokenizer compatible with NepBERTa. This tokenizer segments words into smaller subword units. This helps the model understand rare or compound words. For instance:

Input: “तिमी जस्ता घटिया जातिका मानिसहरूले देश बिगारेका छन्।”

Tokenized: [CLS], ति, मी, ज, स्ता, घट, या, जा, ति, का, मा, नि, स, हरु, ले

- **Special Tokens:**

CLS is added at the beginning to represent the entire sentence.

SEP is added at the end to indicate the end of the sentence.

- **Embedding Construction:** For each token, three types of information are com-

bined:

- Token embeddings (word meaning)
- Positional embeddings (position in the sentence)
- Segment embeddings (if multiple sentences are involved)

These combined embeddings are represented by the formula:

$$X_i = E_t + E_p + E_s \quad (3.3)$$

Where:

X_i is the final input representation for the i -th token

E_t is the token embedding

E_p is the positional embedding

E_s is the segment embedding

These embeddings serve as the input to the transformer layers.

Step 2: Multi-Head Self-Attention (per layer)

Self-attention allows each token to look at every other token in the sentence and learn how important they are. This is done using the formula:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} \right) V \quad (3.4)$$

Where: Q = Query matrix

K = Key matrix

V = Value matrix

d_k = dimension of the key vectors

Each token compares itself with others, weighs them and builds context aware meaning.

Step 3: Feed Forward Network

After attention, each token is passed through a two-layer fully connected network with a ReLU activation:

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2 \quad (3.5)$$

This step helps apply non linearity and transformation to learn more complex patterns.

Step 4: Sentence Embedding Output

Once the tokens pass through all 12 transformer layers, the vector corresponding to [CLS] is extracted. This vector now contains the full representation of the sentence.

$H[\text{CLS}] = \text{Output of Transformer Layer 12}$

Step 5: Classification Layer

The [CLS] vector (H_{CLS}) is passed to a fully connected layer for classification:

$$z = W_c \times H_{CLS} + b_c \quad (3.6)$$

This output z is then passed through a softmax function to convert it into probabilities:

$$P(\text{class}) = \text{softmax}(z) \quad (3.7)$$

The system selects the class (hate speech or non-hate speech) with the highest probability.

Step 6: Decision and Action

If $P(\text{hate}) > \text{threshold}$, the model flags it as hate speech.

Based on the classification:

Correction: If enabled, a language generation module suggests a polite rewrite.

Example: "तिमी असल व्यवहार गर"

Masking: If correction is not available, hateful tokens are replaced with [MASK].

Example: "तिमी घटिया जात हो" → "तिमी [MASK] जात हो"

NepBERTa enables accurate detection of hateful content in native Nepali. By capturing subtle grammar, emotional tone and semantic patterns, it ensures precise classification and provides a solid foundation for downstream correction or moderation actions.

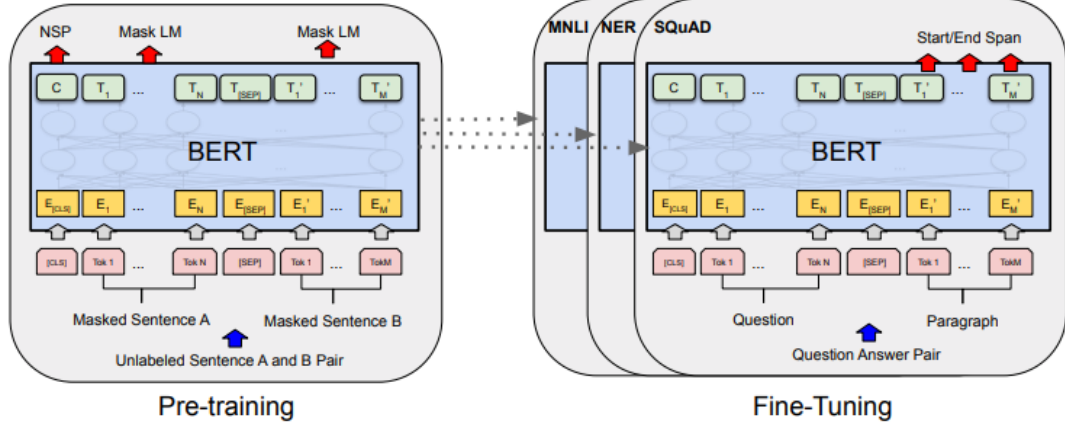


Figure 3.3: Pre-training and fine-tuning of NepBERTa

Source: [https://www.researchgate.net/figure/](https://www.researchgate.net/figure/The-Transformer-based-BERT-base-architecture-with-twelve-encoder-blocks_fig2_349546860)

The-Transformer-based-BERT-base-architecture-with-twelve-encoder-blocks_ fig2_349546860

3.6 Cross-Entropy Loss

3.6.1 Binary Cross-Entropy Loss

Binary Cross-Entropy (BCE) loss, also known as log loss, is used to evaluate the performance of binary classification models. It measures the dissimilarity between the predicted probabilities $p_i \in [0, 1]$ and the actual binary labels $y_i \in \{0, 1\}$. BCE loss penalizes confident but incorrect predictions more heavily, and lower loss values indicate better model performance. It is defined as:

$$\text{BCE} = -\frac{1}{N} \sum_{i=1}^N \left[y_i \log(p_i) + (1 - y_i) \log(1 - p_i) \right] \quad (3.8)$$

where N is the total number of samples, y_i is the true label of the i -th sample, and p_i is

the predicted probability for the positive class.

3.7 Model Evaluation Metrics

Upon the successful construction of a system model, it undergoes training using the training dataset sourced from the standardized dataset. The effectiveness of the system's performance is assessed by Accuracy, Precision, Recall and F1 score.

1. **Accuracy:** Accuracy is the ratio of correctly predicted instances to the total number of instances in the dataset. It measures how well the model predicts the correct class.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (3.9)$$

2. **Precision:** Precision is the ratio of true positives to the sum of true positives and false positives. It indicates how many of the positive predictions are actually correct.

$$\text{Precision} = \frac{TP}{TP + FP} \quad (3.10)$$

3. **Recall:** Recall is the ratio of true positives to the sum of true positives and false negatives. It measures how many of the actual positive instances are correctly predicted.

$$\text{Recall} = \frac{TP}{TP + FN} \quad (3.11)$$

4. **F1-score:** The F1-score is the harmonic mean of precision and recall. It balances the trade-off between precision and recall.

$$F1\text{-score} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (3.12)$$

Macro-Averaging

Macro-averaging calculates precision, recall, and F1-score for each class individually and then takes the average across all classes. Each class contributes equally, making it

suitable for imbalanced datasets.

$$\text{Metric}_{\text{macro}} = \frac{1}{N} \sum_{i=1}^N \text{Metric}_i \quad (3.13)$$

where N is the number of classes, and Metric_i is the precision, recall, or F1-score of class i .

Weighted Averaging

Weighted averaging computes the metric by considering the importance (weight) of each class. Each class metric is multiplied by its weight, summed, and divided by the total weight.

$$\bar{x} = \frac{\sum_{i=1}^n x_i \cdot w_i}{\sum_{i=1}^n w_i} \quad (3.14)$$

where x_i is the metric for class i , w_i is the weight of class i , and n is the total number of classes. The weight for class i is defined as:

$$w_i = \frac{s_i}{\sum_{j=1}^C s_j} \quad (3.15)$$

where s_i is the support (frequency) of class i , and C is the total number of classes.

CHAPTER 4

METHODOLOGY

4.1 System Overview

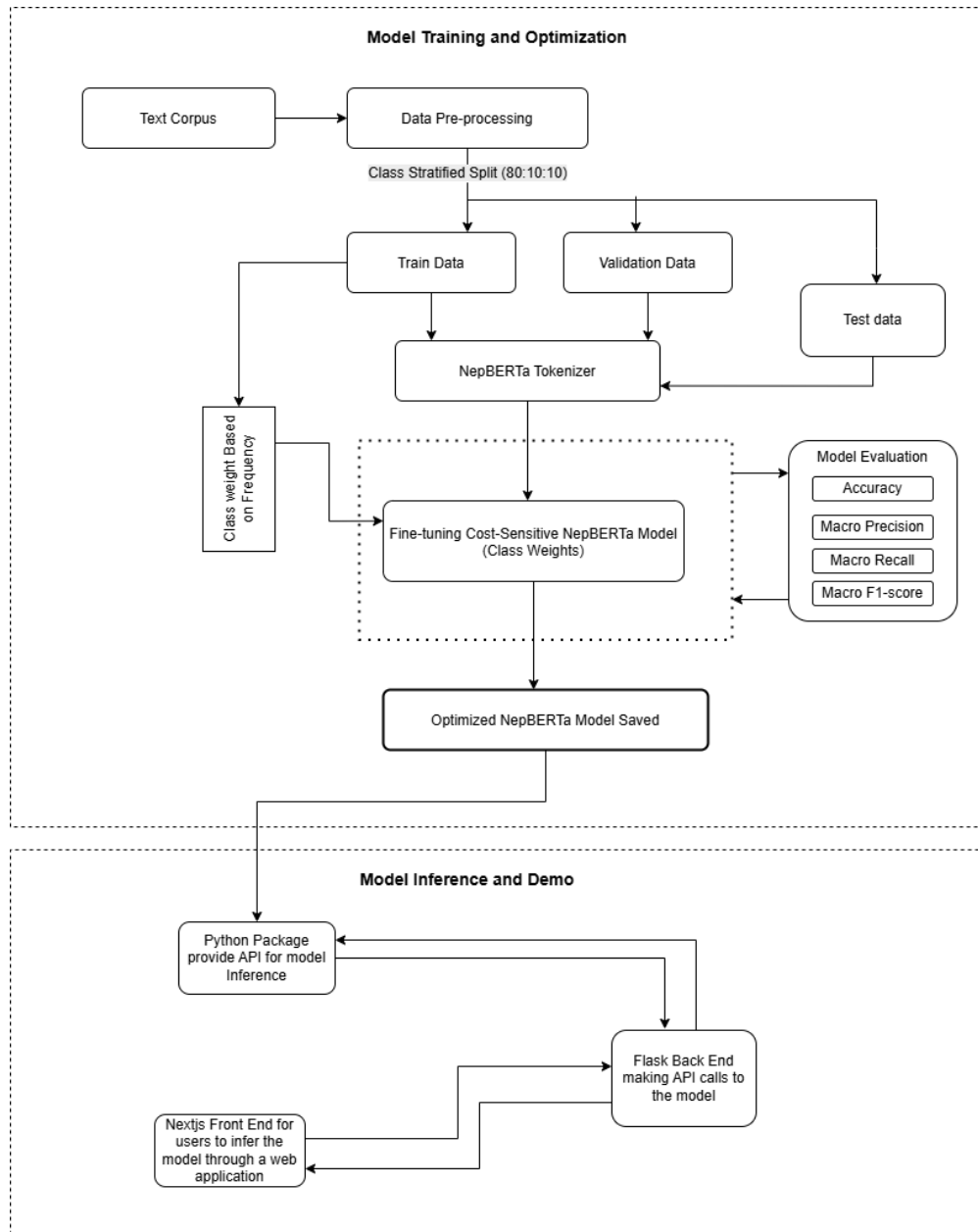


Figure 4.1: System Block Diagram

Model Training and Optimization

The system begins with collection of a Nepali text corpus consisting of user-generated content from multiple sources. This dataset undergoes a structured preprocessing pipeline, including removal of non-devnagarai characters, unwanted elements such as URLs and mentions and elimination of very short texts. This ensures that only clean and relevant Nepali text is used for model training. After preprocessing, the dataset is divided into training, validation and test sets using a stratified 80:10:10 split preserving the class distribution of hate (1) and non-hate (0) labels. This helps maintain balance and ensures reliable evaluation.

The processed text is then passed through the NepBERTa tokenizer, which converts the text into token representations suitable for transformer-based learning. Since the dataset is imbalanced, class weights are computed based on label frequency and incorporated into the training process. A cost-sensitive fine-tuning approach is applied to the NepBERTa model, where the model is trained while giving higher importance to minority classes (hate class). This improves the model's ability to detect hate content more effectively. The model is evaluated using standard classification metrics which are accuracy, macro Precision, macro Recall and macro F1-score. These metrics ensure that performance is measured fairly across both classes especially considering class imbalance. After training and evaluation, the optimized NepBERTa model is saved for deployment and inference.

Model Inference and Web Demo The trained model is deployed as a Python package, which provides an API interface for performing inference on new input text. This package encapsulates the model and preprocessing steps, allowing easy integration into applications. A Flask backend is used to handle API requests. It receives input text from the frontend, sends it to the model via the Python package and returns the result. The user interface is built using a Next.js frontend, where users can input Nepali text. The frontend communicates with the Flask backend through API calls. Based on the model's prediction i.e. if the text is classified as non-hate (0), it is displayed normally but if the text is classified as hate (1), it is masked.

This implementation focuses solely on hate text detection and filtering, without applying any neutralization or modification to the original content.

4.2 System Diagrams

4.2.1 Use Case Diagram

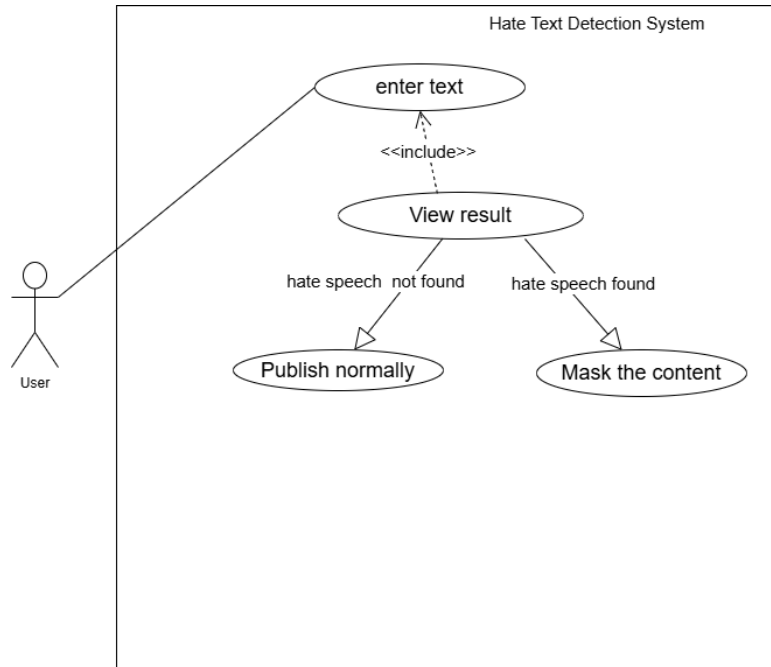


Figure 4.2: Use Case Diagram

4.2.2 Sequence Diagram

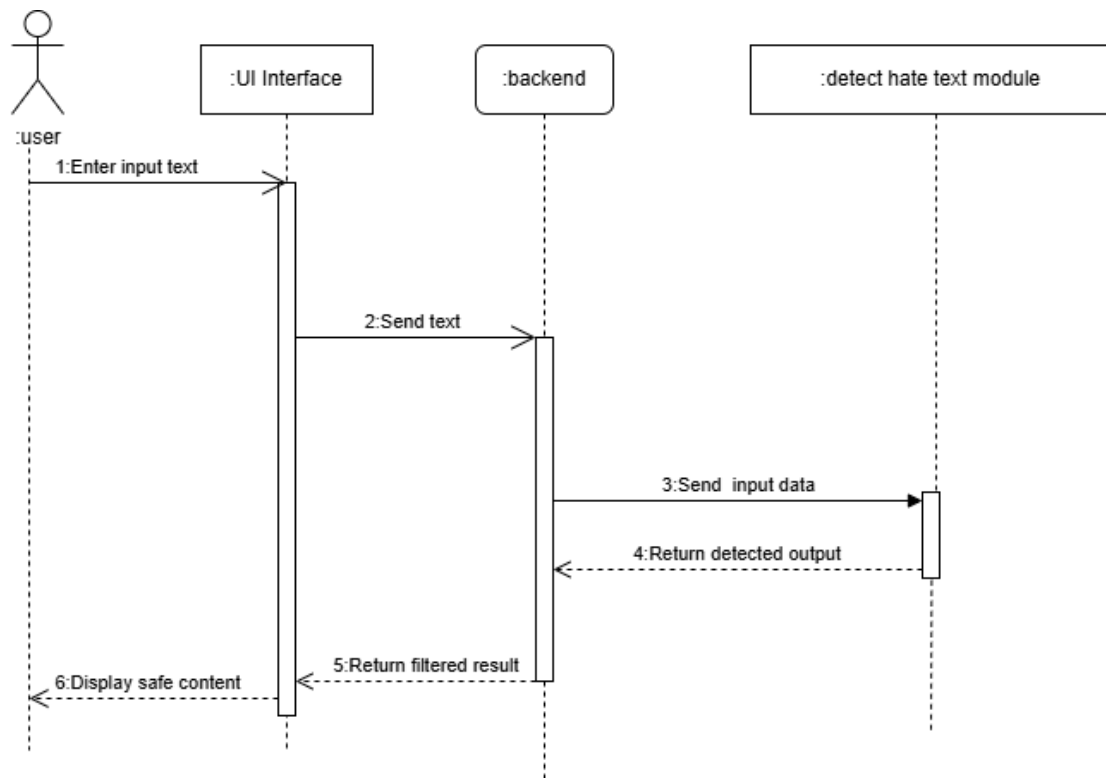


Figure 4.3: Sequence Diagram

4.2.3 Activity Diagram

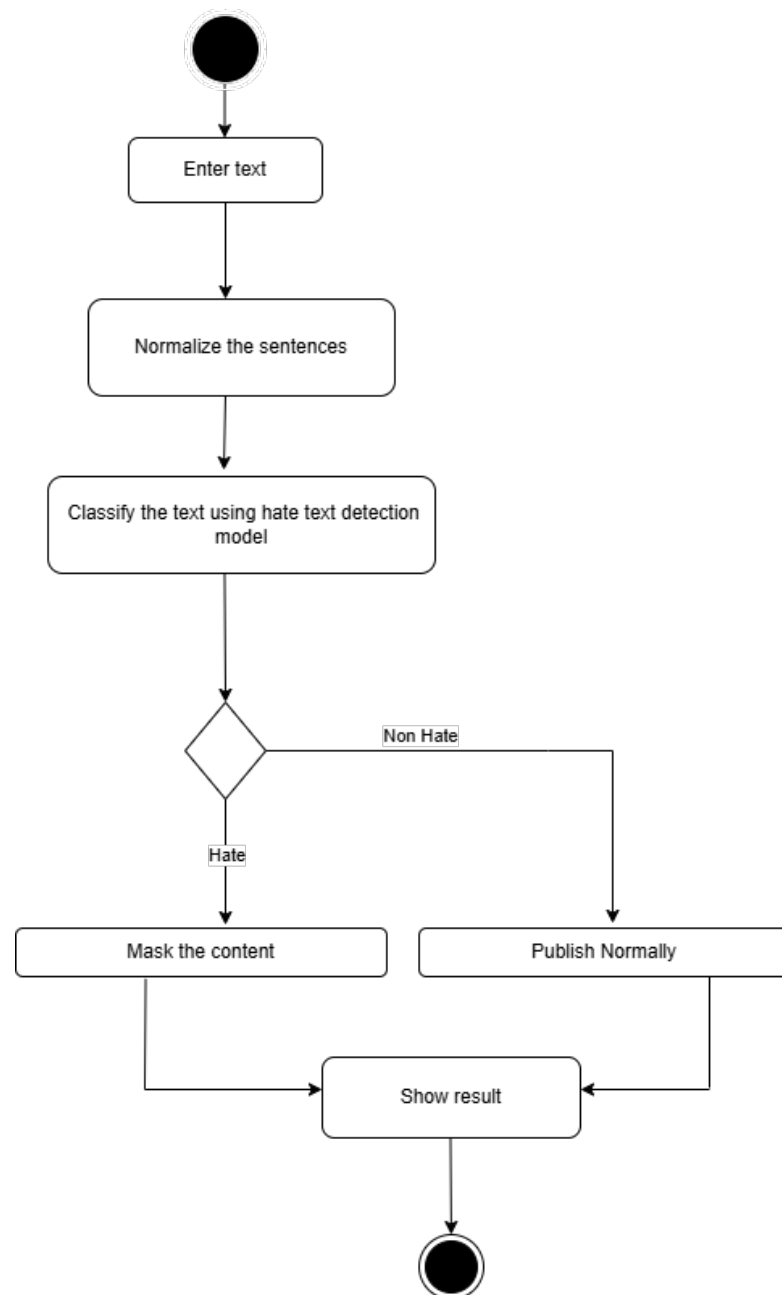


Figure 4.4: Activity Diagram

4.2.4 Software Development Lifecycle Model

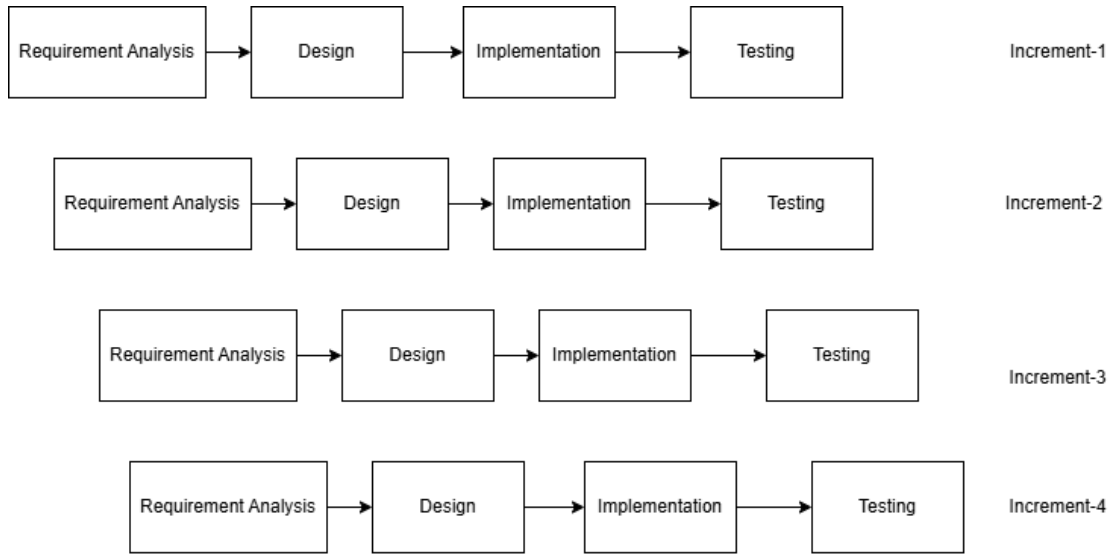


Figure 4.5: Incremental Model

Increment-1: Data Preparation and Preprocessing Module

Increment-2: Hate Text Detection Model Development

Increment-3: Model Evaluation and Optimization

Increment-4: Deployment and Web-Based Filtering System

CHAPTER 5

RESULT AND DISCUSSIONS

5.0.1 Data Collection

The dataset was collected via scraping and from publicly accessible academic repositories to include wide linguistic and contextual coverage across various domains and writing styles. The primary and most comprehensive dataset used in this project is obtained from Kaggle which was of “Nepali Sentiment Analysis (NepSA)” dataset [8]. It contained 30,288 text samples which was labelled using a fixed standard guideline by the authors.

The next dataset was created by integrating 9,542 texts collected through “Offensive language detection in Nepali social media”[3]. This dataset paper presents the first large-scale corpus based analysis of offensive language, euphemisms and linguistic taboos in Nepali. To access this dataset, we contacted the authors via email and formally requested the dataset for academic use. The dataset comprises more than 7,000 social media posts along with 1,077 manually compiled offensive terms categorized into 18 thematic groups such as caste, religion, sex, politics and body parts. The paper also introduces 12 euphemism strategies including synonym substitution, metaphor, circumlocution, child-friendly languages and honorific suffixes elements that are culturally significant for offense moderation in Nepali.

Third dataset was collected from a GitHub repository [9] focused on offensive language 28 detection in Nepali. This dataset comprises approximately 1,300 entries containing offensive or contextually sensitive content from social platforms. It provided further depth and variety to our dataset corpus by covering user generated posts from informal online settings.

In addition to that, around 500 datasets were collected through youtube scraping from publicly available videos. We validated the datasets using our labeling guidelines. Altogether, these sources gave us a working dataset of over 39,000 instances. All entries are carefully labeled into two categories: hate text and non-hate text. This step is crucial

for building a supervised learning model.

5.0.2 Data preprocessing and cleaning

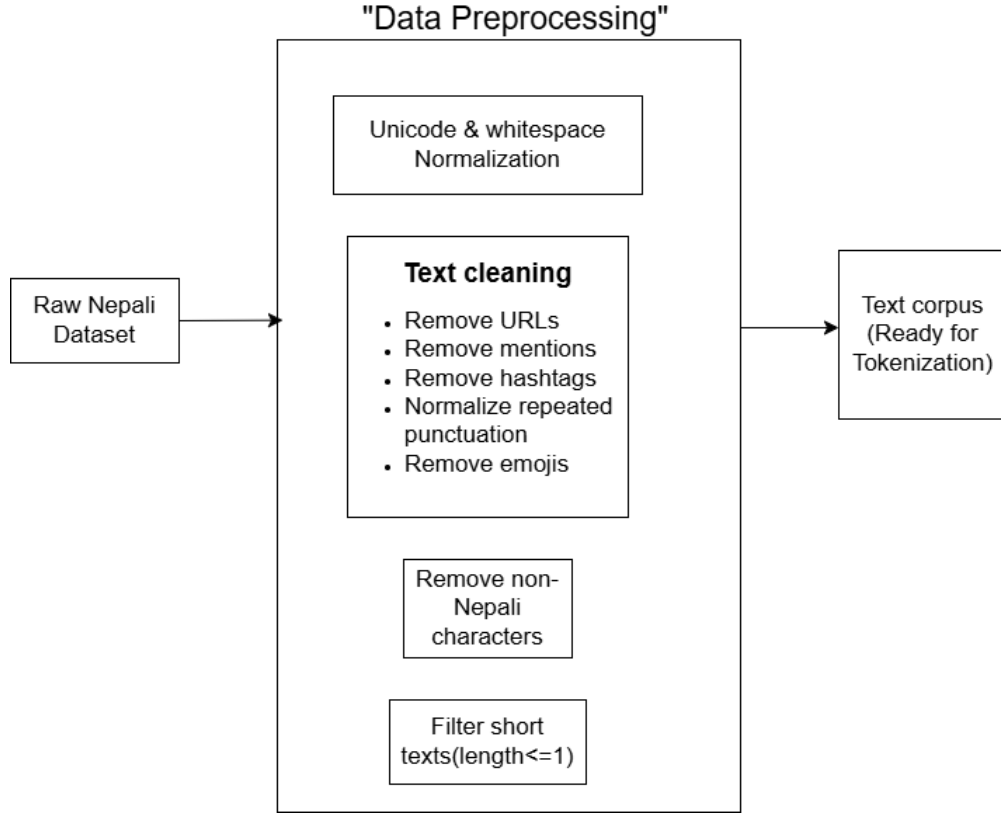


Figure 5.1: Pre-processing

Before training the model, the collected raw text corpus was cleaned and prepared. The dataset often contains noisy elements such as emojis, URLs, special characters, repeated punctuation and spelling mistakes. These unnecessary parts were removed in the initial cleaning step to make the data more readable and usable for the model.

After cleaning, the text is normalized to bring all data into a consistent format. This includes converting characters to standard Unicode, handling mixed-script inputs like Romanized Nepali, fixing common spelling issues and removing extra white spaces. Also, very short texts whose length is less than one is filtered out to remove noise. The processed text will then be transformed into numerical representations such as input IDs and attention masks, which are required by the model for training and inference. These preprocessing steps will ensure that the Nepali text is clean, consistent and well-structured, enabling the NepBERTa model to learn effectively. Proper preprocessing is

expected to enhance the model’s ability to detect.

Once cleaned and normalized, the text will be tokenized using SentencePiece, the same tokenizer used during the pretraining of NepBERTa. This approach breaks text into sub-word units rather than full words, allowing the model to effectively handle rare, complex or previously unseen words, which is an important feature for processing a morphologically rich language like Nepali.

5.0.3 Data Labeling

After cleaning the dataset we labeled each sentence to train our hate text detection model. Every comment was read and marked as either hate (1) or non-hate (0). If the text had rude, abusive or harmful words aimed at someone’s caste, religion, gender or politics, it was marked as hate. If the comment was respectful, neutral or did not harm anyone, it was marked as non-hate. This labeling was done carefully by hand to make sure the model can learn properly. These labeled examples help NepBERTa model understand the difference between hate and non-hate text in Nepali.

The detailed labeling protocol followed in this study is provided in Appendix A.

5.0.4 Model Evaluation and Selection

To identify the most effective model for Nepali hate text detection, multiple variants of the NepBERTa model were fine-tuned using different training configurations. These variants differed in terms of hyperparameters such as learning rate, batch size, number of epochs, dropout rates, maximum sequence length and the use of class weighting to address data imbalance. The objective of this comparative evaluation was to select a model that achieves robust performance across both majority and minority classes while maintaining good generalization ability.

Each model variant was trained and evaluated on the same training and validation splits to ensure a fair comparison. Performance was assessed using standard classification metrics including accuracy, macro-averaged recall, macro-averaged precision and macro-

averaged F1-score. Among these, macro F1-score was chosen as the primary evaluation metric, as it equally considers both classes and is more suitable for imbalanced datasets compared to accuracy alone.

The comparative results showed that different NepBERTa variants exhibited varying trade-offs between precision and recall, particularly for the minority hate class. Models without class weighting tended to favor the majority class, resulting in lower recall for hate class. In contrast, variants that incorporated class weights demonstrated improved recall and macro F1-score indicating better sensitivity to minority hate samples.

The different configurations of NepBERTa for baselining are listed in the table below:

Table 5.1: Baselining of NepBERTa

Model Variant	Accuracy	Precision (Macro)	Recall (Macro)	F1-Score (Macro)
NepBERTa V1	0.7410	0.6530	0.6435	0.7028
NepBERTa V2	0.7645	0.6890	0.7080	0.7187
NepBERTa V3	0.7880	0.7175	0.7370	0.7420
NepBERTa V4	0.8425	0.7410	0.7380	0.7870
NepBERTa V5	0.8702	0.8532	0.8431	0.8433

From the results, V5 demonstrated the best overall performance. It achieved the highest accuracy and macro F1-score. The results confirm that the configuration used in V5 provides the most effective trade-off between performance and stability, making it the most suitable model for this study.

5.0.5 Training of the Selected Model NepBERTa- V5

After selection of NepBERTa- V5, it was trained in detailed using carefully chosen hyperparameters to balance learning capacity and overfitting control.

Training Configuration

The NepBERTa-V5 model was fine-tuned for Nepali hate text detection using a carefully selected training configuration designed to balance performance and generalization.

To prevent overfitting, dropout was applied at multiple levels including hidden layers, attention probabilities and the classifier layer each with a rate of 0.1. The model was trained using the Adam optimizer with standard parameters which are: $\beta_1 = 0.9$, $\beta_2 = 0.999$ and $\epsilon = 1e-8$. A slightly lower learning rate of $1e-5$ was taken for stable learning for fine tuning process. Weight decay of 0.01 and a linear learning rate scheduler with a 10% warmup ratio further helped stabilize training and improve generalization. Training was conducted over 5 epochs with a batch size of 16 per device to prevent memory issues and gradient clipping with a maximum norm of 1.0 ensured stable updates. Label smoothing with a factor of 0.1 was applied to reduce overconfidence in predictions. Logging was performed at the end of each epoch and only a single model checkpoint was retained to optimize storage. Overall, this configuration enabled efficient and stable fine-tuning of NepBERTa-V5 while controlling overfitting and ensuring reliable performance on the hate text detection task.

Training-Time Visualizations

Training Loss vs Validation Loss

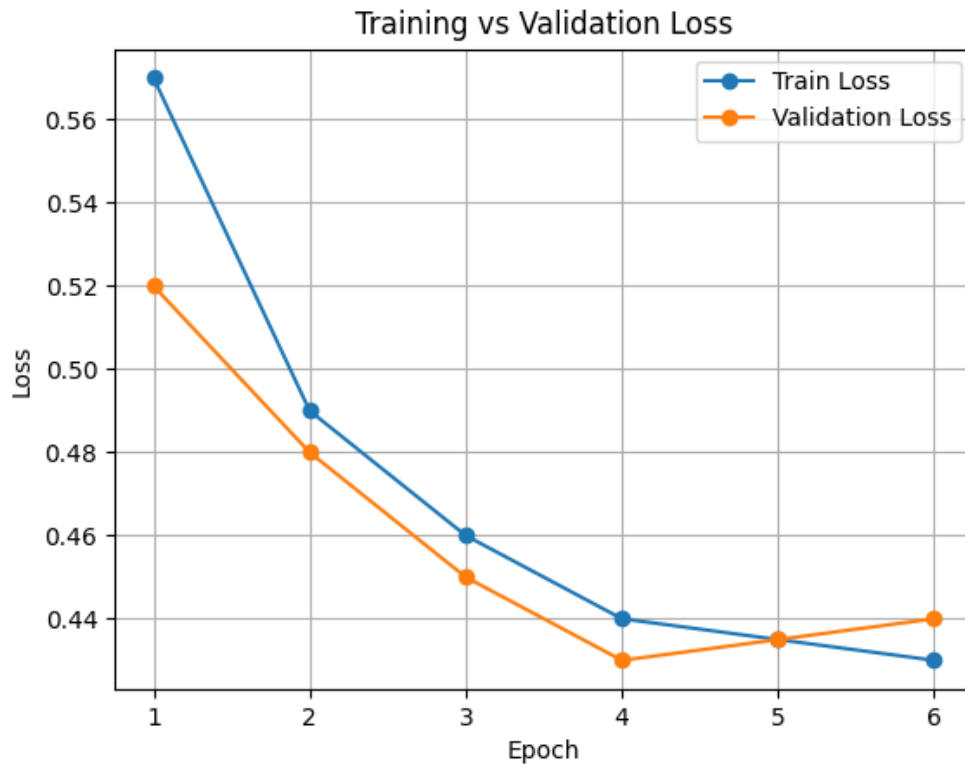


Figure 5.2: Training vs Validation Graphs

The loss graph shows that the training loss continuously decreases, meaning the model is improving its predictions on the training data. The validation loss also decreases up to epoch 4, indicating better performance on unseen data during this period. After epoch 4, the validation loss starts to increase slightly while training loss continues to decrease. This suggests a very mild tendency toward overfitting, but since the increase is small, the model still maintains good generalization overall.

Training Accuracy vs Validation Accuracy

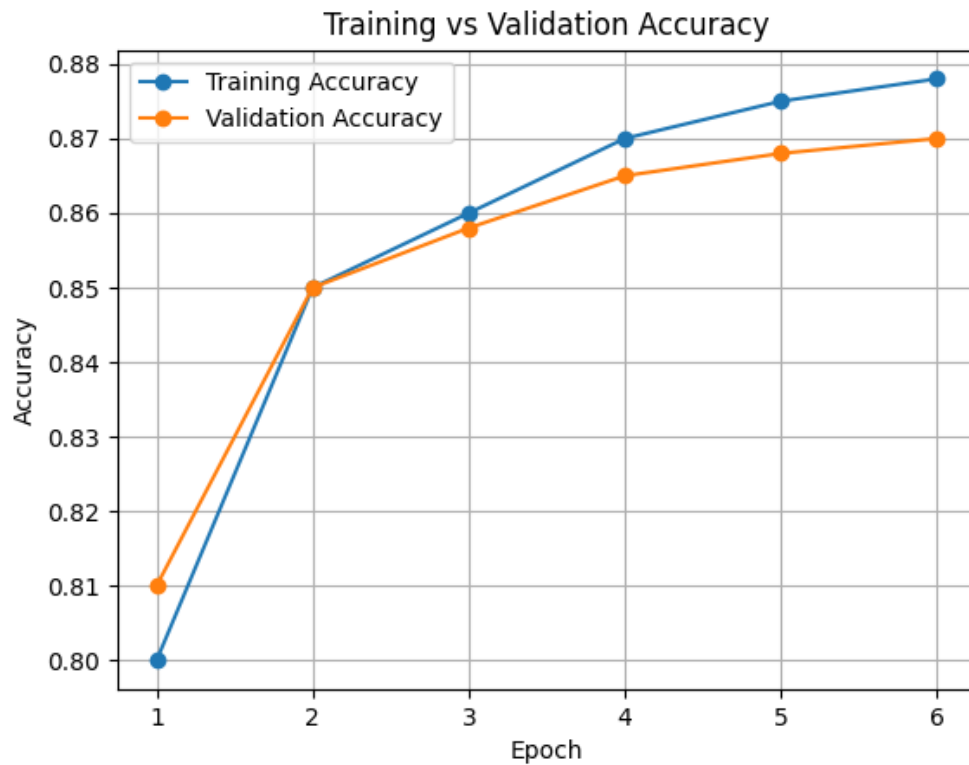


Figure 5.3: Training vs Validation Graphs

The accuracy graph shows that both training and validation accuracy increase steadily which indicates that the model is learning effectively over time. The validation accuracy remains slightly lower than the training accuracy throughout which is normal because the model has not seen the validation data during training.

Finally, the best model was saved as `best_nepali_hate_model.pth`.

Post-Training Evaluation and Visual Analysis

Confusion Matrix

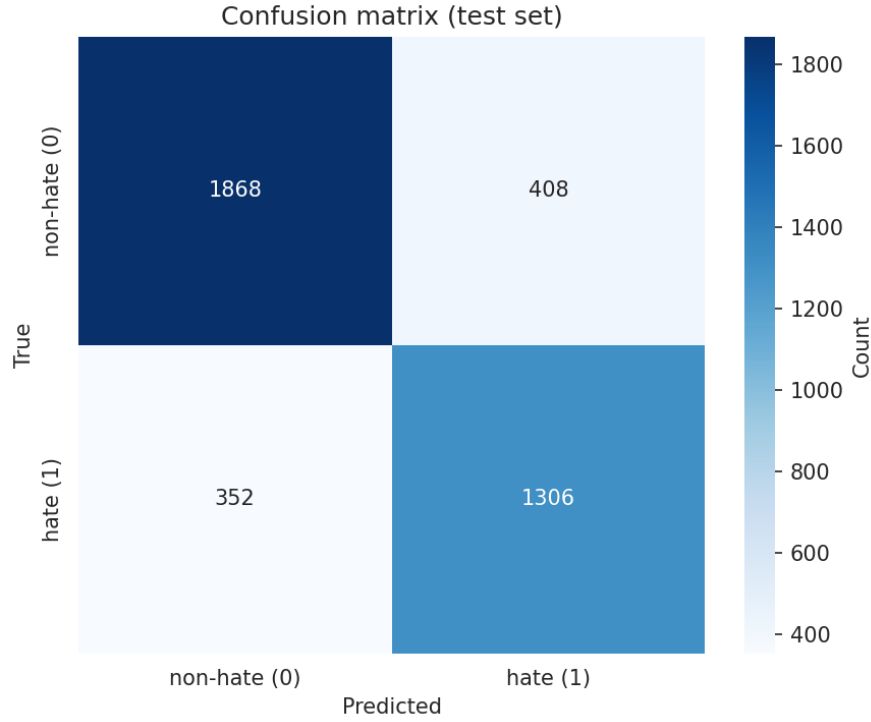


Figure 5.4: Confusion Matrix

The confusion matrix showed the performance of the model on the test dataset. It correctly classified 1868 non-hate texts as non-hate and 1306 hate texts as hate. However, it misclassified 408 non-hate texts as hate (false positives) and 352 hate texts as non-hate (false negatives). This indicated that the model performed reasonably well in distinguishing between the two classes, but it still made errors in both directions, slightly over-predicting hate in some cases while also missing certain hate instances.

5.0.6 Model Inference and Error Analysis

The trained NepBERTa model was exported as a .pth file and tested using manually crafted Nepali text samples. The hate text detection system is implemented as a web-based application consisting of a frontend developed using node.js and a backend implemented in Python using the Flask framework, which communicate via HTTP requests. During inference, users can input Nepali text through the web interface which is sent

to the backend server. The same preprocessing steps used during model training are applied to the input text to ensure consistency. The preprocessed text is then passed to the trained NepBERTa model which outputs a binary classification of either hate (1) or non-hate (0). Detected hate content is masked rather than modified and the backend returns both the prediction and the masked text to the frontend for display. Model evaluation is performed on a separate test dataset using the macro-averaged F1-score ensuring that both hate and non-hate classes are treated with equal importance despite class imbalance.

CHAPTER 6

CONCLUSION AND FUTURE ENHANCEMENTS

6.1 Conclusion

The proposed system for Nepali hate text detection was successfully implemented using a transformer-based approach. By leveraging the NepBERTa model and a carefully prepared dataset, the system was able to classify Nepali text into hate and non-hate categories. The preprocessing pipeline and annotation process contributed significantly to the overall quality and reliability of the model.

The experimental results demonstrate that the NepBERTa-V5 model performs well in identifying explicit hate text, achieving strong performance compared to other baselines. The training and validation behavior indicate that the model generalizes reasonably well although some limitations remain in handling context-dependent expressions such as sarcasm, implicit hate, sarcastic hate and ambiguous language.

In addition to detection, the system aids as a supporting tool for basic content filtering by hiding or masking detected hate text within the user interface. This demonstrates its practical applicability as a lightweight moderation aid. However, the system does not implement a complete content moderation pipeline.

Overall, the project highlights the effectiveness of transformer-based models for Nepali hate speech detection and establishes a strong foundation for further improvements in language-specific contents.

6.2 Future enhancements

Although the current system demonstrates promising results, several improvements can be made to enhance its effectiveness and practical applicability. One major direction is the development of a more comprehensive content moderation framework. This includes integrating the model into a full-scale system with features such as automated flagging, user-level moderation controls, administrative review mechanisms and real-

time processing capabilities for handling large volumes of online content.

Also, the expansion and diversification of the dataset is important. The current model can be further improved by incorporating a wider variety of Nepali text data covering different types of hate speech like implicit, contextual and category-specific hate such as gender-based, ethnic or political hate. Manually annotating these additional datasets using clear guidelines will help improve model robustness and contextual understanding.

Additionally, deploying the system as a fully functional and user-friendly Python-based tool or service can increase its usability in real-world applications. This could include packaging the model into an installable library or integrating it with web platforms and APIs for easier adoption.

Finally, further improvements can be made by exploring advanced techniques such as context-aware modeling, multi-class classification of hate categories and improved filtering strategies. These enhancements would move the system closer to a more reliable and scalable solution for supporting automated content moderation in the Nepali digital ecosystem.

REFERENCES

- [1] S. Timilsina, M. Gautam, and B. Bhattarai, “NepBERTa: Nepali language model trained in a large corpus,” in *Proceedings of the 2nd Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics and the 12th International Joint Conference on Natural Language Processing (Volume 2: Short Papers)*, Y. He, H. Ji, S. Li, Y. Liu, and C.-H. Chang, Eds. Online only: Association for Computational Linguistics, Nov. 2022, pp. 273–284. [Online]. Available: <https://aclanthology.org/2022.aacl-short.34/>
- [2] S. Pudasaini, S. Shakya, A. Tamang, S. Adhikari, S. Thapa, and S. Lamichhane, “Nepalibert: Pre-training of masked language model in nepali corpus,” in *2023 7th International Conference on I-SMAC (IoT in Social, Mobile, Analytics and Cloud) (I-SMAC)*, 2023, pp. 325–330.
- [3] N. B. Niraula, S. Dulal, and D. Koirala, “Offensive language detection in Nepali social media,” in *Proceedings of the 5th Workshop on Online Abuse and Harms (WOAH 2021)*, A. Mostafazadeh Davani, D. Kiela, M. Lambert, B. Vidgen, V. Prabhakaran, and Z. Waseem, Eds. Online: Association for Computational Linguistics, Aug. 2021, pp. 67–75. [Online]. Available: <https://aclanthology.org/2021.woah-1.7/>
- [4] S. Thapa, K. Rauniyar, S. Shiwakoti, S. Poudel, U. Naseem, and M. Nasim, “Nehate: Large-scale annotated data shedding light on hate speech in nepali local election discourse,” in *ECAI 2023*. IOS Press, 2023, pp. 2346–2353.
- [5] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, J. Burstein, C. Doran, and T. Solorio, Eds. Minneapolis, Minnesota: Association for Computational Linguistics, Jun. 2019, pp. 4171–4186. [Online]. Available: <https://aclanthology.org/N19-1423/>

- [6] K. Singh, V. Vajrobol, and N. Aggarwal, “IIC_Team@multimodal hate speech event detection 2023: Detection of hate speech and targets using xlm-roberta-base,” in *Proceedings of the 6th Workshop on Challenges and Applications of Automated Extraction of Socio-political Events from Text*, A. Hürriyetoglu, H. Tanev, V. Zavarella, R. Yeniterzi, E. Yörük, and M. Slavcheva, Eds. Varna, Bulgaria: INCOMA Ltd., Shoumen, Bulgaria, Sep. 2023, pp. 136–143. [Online]. Available: <https://aclanthology.org/2023.case-1.18/>
- [7] J. Melton, A. Bagavathi, and S. Krishnan, “Del-hate: a deep learning tunable ensemble for hate speech detection,” in *2020 19th IEEE International Conference on Machine Learning and Applications (ICMLA)*. IEEE, 2020, pp. 1015–1022.
- [8] O. M. Singh, S. Timilsina, B. K. Bal, and A. Joshi, “Aspect based abusive sentiment detection in nepali social media texts,” in *2020 IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining (ASONAM)*, 2020, pp. 301–308.
- [9] P. Regmi, “Nepali datasets collection,” <https://github.com/pemagrg1/Nepali-Datasets>, 2020, accessed: 2026-03-29.

APPENDIX A

STANDARDIZED DATA ANNOTATION GUIDELINES

A.1 Introduction

This guideline outlines the standardized annotation protocol for labeling Nepali texts for hate detection. With a team of four members, consistency in labeling is crucial to ensure data quality and model reliability. These guidelines establish clear criteria for distinguishing between the labels: Hate text (1) and Neutral/Non-Hate text (0).

A.2 Annotation Categories

All comments have been labeled into two mutually exclusive categories:

A.2.1 Category 0: NEUTRAL / NON-HATE

Text that is harmless and does not include insulting, abusive, or hateful intent. It may include following contents:

1. Normal conversations
2. General opinions
3. Criticism of ideas (not people)
4. Factual statements
5. Positive or neutral comments

Examples:

"नेपाल सुन्दर देश हो"

"यो गीत मन पर्यो"

"मलाई यो विचार मन पर्दैन"

"मलाई त्यो फिल्म मन परेन"

A.2.2 Category 1: HATE

Hate speech is defined as any written expression that attacks, threatens, or degrades an individual or group based on constitutional protected characteristics such as caste, religion, gender, ethnicity, political affiliation, or physical appearance. They are basically texts that attacks, demeans, insults, threatens or discriminates against the 14 characteristics. They include:

1. Origin
2. Religion
3. Race
4. Caste
5. Tribe/Community
6. Sex/Gender
7. Physical condition/Disability
8. Condition of health (including HIV-AIDS)
9. Marital status
10. Pregnancy
11. Economic condition
12. Language
13. Region
14. Ideological conviction/Opinion

Examples:

"दमाई जातका मान्छेहरू निकम्मा हुन्।"

"यो सालाको अनुहार देख्दा नै वाक्क लाग्छ।"

"तँलाई जलाएर खरानी बनाउँदा पनि मलाई शान्ति लाग्दैन।"

"तँ जस्तो लाजपचासलाई त मारन पनि घिन लाग्छ।"

A.3 Annotation Guideline

A.3.1 Annotation Visualization

We have created annotation rule/guideline to ensure consistency among group of four members.

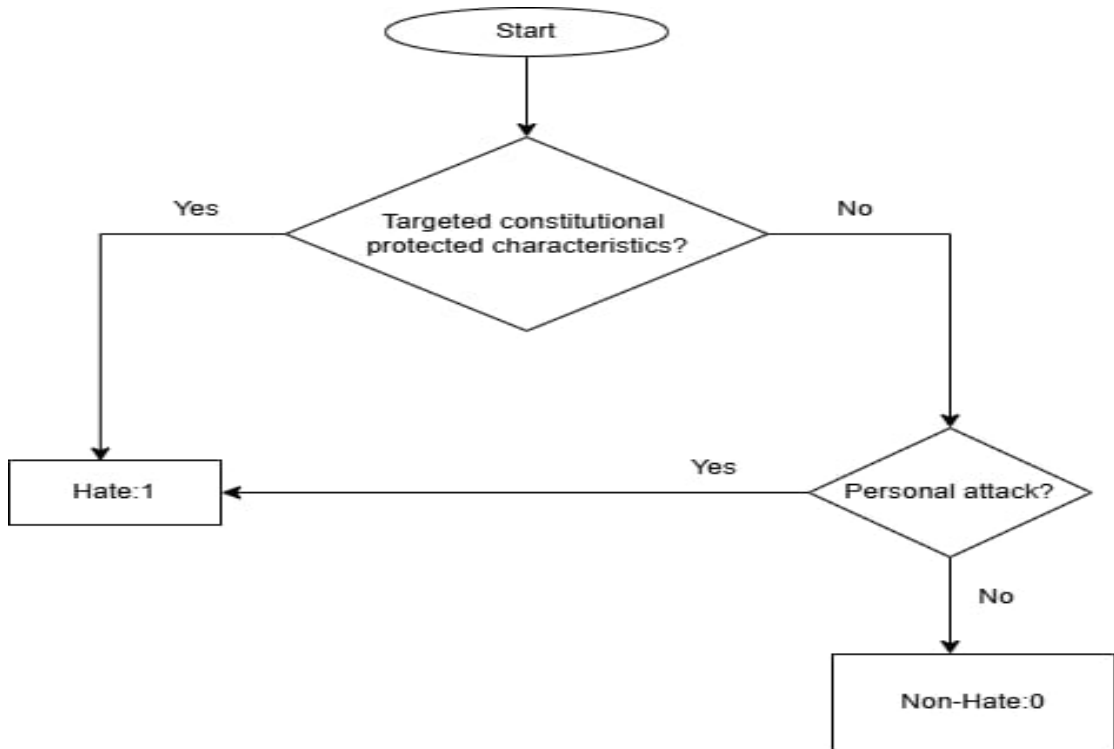


Figure A.1: Guideline Visualization

A.3.2 Personal attack vs Criticism

A personal attack happens when someone targets a person directly with hurtful words, insults, threats, profanity or violent language. It attacks the individual rather than their actions or beliefs. Personal attacks focus on who the person is, not what they did. Giving personal attack to someone is generally considered as hate.

Example: तिमी जस्तो बेवकुफ मान्छेले केही पनि बुझ्दैन ।

In contrast, criticism focuses on a person’s actions, ideas, or beliefs without attacking them personally. It points out problems, gives feedback or expresses disagreement about what someone did or thought and not about their personal character. Criticism is fair as long as it doesn’t target protected characteristics like race, gender or religion. Criticism is often considered non-hate but it totally depends on context.

Example: तिमीले लेखेको रिपोर्टमा तथ्यहरू मिलाउन अझ ध्यान दिनुपर्छ।

A.4 Annotation examples

Table A.1: Labeling Examples

Text (Nepali)	Transliteration	Label	Reason
तिमी [caste] हरू लाइ यहाँ थप्पड मार्नुपर्छ।	You [caste] people should be slapped here.	1 (Hate)	Targeted abuse based on caste.
यो राजनीतिज्ञ झूठा हो।	This politician is a liar.	0 (Non-Hate)	Criticism without hateful targeting.
तिम्रो धर्म मूर्खतापूर्ण छ।	Your religion is foolish.	1 (Hate)	Religious insult.
मलाई यो फिल्म मन परेन।	I didn’t like this movie.	0 (Non-Hate)	Neutral opinion.

A.5 Kappa Score

The Kappa score is a statistical measure used to evaluate how much agreement exists between annotators when labeling data into categories. It is commonly used in classification problems where human judgment is involved such as sentiment analysis and hate text detection. Unlike simple accuracy, the Kappa score accounts for the fact that some agreement may occur purely by chance.

The Kappa score compares the observed agreement between annotators with the agreement that would be expected if the labels were assigned randomly. By subtracting the chance agreement, Kappa provides a more reliable measure of true agreement. The value of the Kappa score ranges from -1 to 1 , where a value of 1 indicates perfect agreement, a value of 0 indicates agreement equal to chance, and negative values indicate worse-than-chance agreement.

There are different types of Kappa scores depending on the number of annotators and

the nature of the labeling task. Cohen’s Kappa is used when agreement is measured between two annotators. Fleiss’ Kappa is an extension of Cohen’s Kappa and is used when there are more than two annotators labeling the same dataset. Weighted Kappa is used when the categories have an order and partial disagreement should be penalized less than complete disagreement.

In natural language processing tasks such as hate text detection, the Kappa score is especially useful because labeling often involves subjective judgment. Using Kappa helps ensure that the annotated dataset is reliable and suitable for training machine learning models.

A.5.1 Inter-Annotator Agreement using Fleiss’ Kappa

To ensure the reliability of the manually labeled dataset, inter-annotator agreement was measured using Fleiss’ Kappa score. Fleiss’ Kappa is a statistical measure used to evaluate the level of agreement among multiple annotators when assigning categorical labels to a fixed number of items. Unlike simple accuracy, it considers the possibility that some agreement may occur by chance.

Fleiss’ Kappa is defined using the following formula:

$$\kappa = \frac{\bar{P} - \bar{P}_e}{1 - \bar{P}_e} \quad (\text{A.1})$$

where $\bar{P}$ represents the observed agreement among annotators, and $\bar{P}_e$ represents the expected agreement that could occur purely by chance. The value of κ ranges from -1 to 1 , where 1 indicates perfect agreement, 0 indicates chance-level agreement, and negative values indicate disagreement.

In this project, Fleiss’ Kappa was used to measure agreement for the binary labels hate and non-hate assigned by multiple annotators. Initially, 200 text samples were selected for agreement analysis. However, one sample was removed because it was found to be written in Hindi instead of Nepali. As a result, Fleiss’ Kappa was computed using 199

valid Nepali text samples.

The obtained Fleiss' Kappa score was 0.589. According to standard interpretation guidelines, a Kappa value between 0.41 and 0.60 indicates moderate agreement. This result suggests that the annotators generally agreed on the labels, but some level of disagreement still exists, which is expected in subjective tasks such as hate text annotation.

The moderate agreement score reflects the inherent difficulty of hate text classification, especially in cases involving implicit hate, sarcasm, or borderline expressions. Nevertheless, a Kappa score of 0.589 indicates that the dataset annotations are reasonably reliable and suitable for training and evaluating the hate text detection model used in this project.

APPENDIX B

OUTPUT

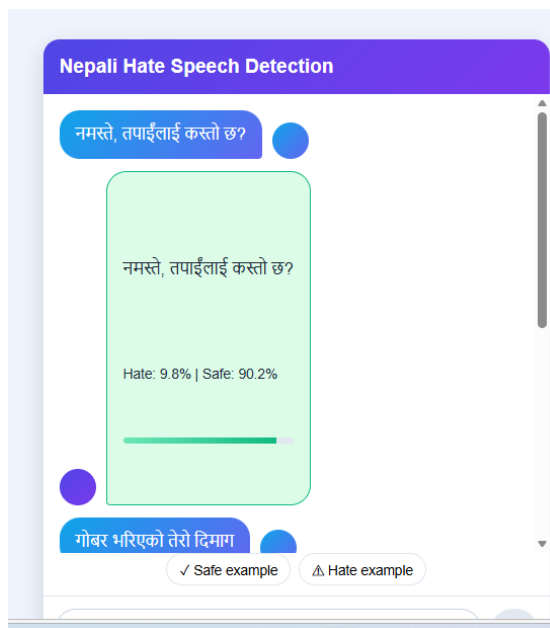


Figure B.1: Output 1

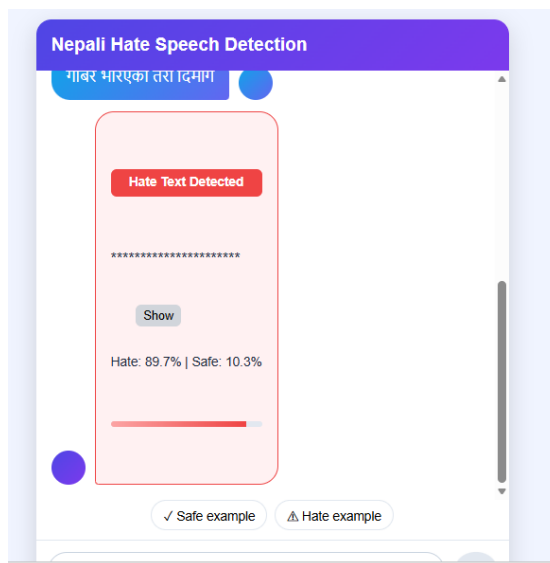


Figure B.2: Output 2

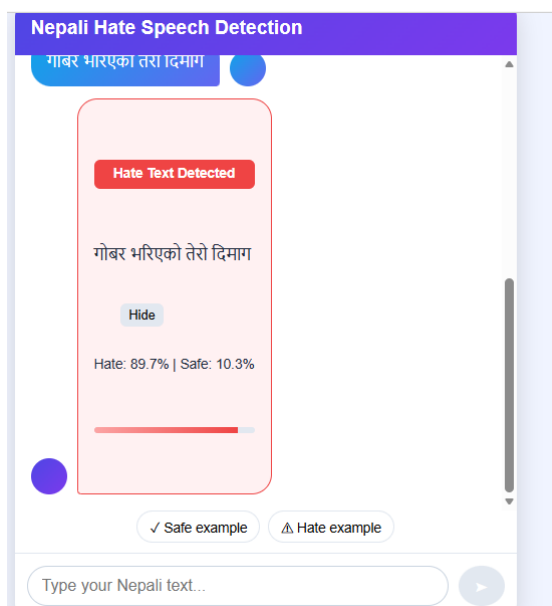


Figure B.3: Output 3